

L&D INTERNSHIP COHORT - 15-DAY PROGRAM

AI-Powered Full-Stack Product Scaffolding

A 15-Day Vibe-Coding Curriculum for Teams from AI, CSE, IoT, and Cybersecurity

15-Day Daywise Schedule

Day	Curated Topic & Sub-topics	First Principles & Value Add
PART I: CORE WORKSPACE SETUP & VIBE-CODING (DAYS 1-4)		
Day 1	Program Intro & LaaS Platform Guide - Cohort onboarding & program structure - Signing up on the LaaS portal - Remote desktop connection and session launching - File upload/download and ZFS storage limits	Onboards interns directly onto local supercomputing hardware. They learn to book, launch, and configure remote RTX 5090 nodes without configuration delays.
Day 2	Vibe-Coding & Antigravity Setup - Understanding the 'Vibe-Coding' paradigm shift - Downloading & installing the Antigravity developer environment - Workspace configurations & environment variables - Running basic agent commands & background checks	Establishes immediate developer velocity by preparing AI pair-programming interfaces and setting up the core workspace shell.
Day 3	Product Dev Workflow & Harness	Teaches interns how to run rapid prototyping workflows, letting AI generate boilerplate while engineers focus on

	Setup ▪ AI-assisted product prototyping loops ▪ Creating file structures & staging folders ▪ Setting up the Agent Harness pipeline ▪ Best practices: prompt-compile loops, validation cycles	architectural verification.
Day 4	Context Engineering: Rules & Skills ▪ Writing custom workspace instructions ▪ Creating specific .qoder rules ▪ Defining repository-specific skills ▪ Managing model context limits & prompt sizes	Ensures the AI assistant conforms strictly to the team's coding guidelines, database standards, and project parameters.
PART II: OPERATING SYSTEMS, LINUX & GIT (DAYS 5-6)		
Day 5	OS Concepts & Linux Operations ▪ Process execution, CPU/RAM isolation, thread monitoring ▪ Linux Filesystem Hierarchy (FHS) & standard paths ▪ File permissions (chmod, chown) ▪ Pipes (), stdout, stdin streams & basic automation	Removes reliance on graphical interfaces, training students in backend process management and remote server troubleshooting.
Day 6	Team Git Collaboration Workflows ▪ Git internals: staging, commits, branches & blobs ▪ Feature-branch flow, opening Pull Requests (PRs) ▪ Review workflows & code merge protocols ▪ Resolving merge conflicts & git rebase	Critical for collaborative success, letting 4-member teams build, test, and merge components in parallel without breaking the main codebase.
PART III: RELATIONAL & DOCUMENT DATABASES (DAYS 7-8)		
Day 7	Relational Databases & SQL Schema Design ▪ ACID transactions & transactional reliability ▪ SQL schema design: PKs, FKs, and tables normalization	Ensures transactional data integrity, teaching students how to design database layouts that support high-concurrency read/write operations.

	▪ B-Tree index lookups for query optimization ▪ PostgreSQL database client setups on LaaS	
Day 8	Non-Relational Databases: MongoDB ▪ CAP Theorem: Consistency vs. Availability vs. Partitioning ▪ JSON document storage vs. tabular relational rows ▪ Designing unstructured collections ▪ Connecting backend API code to MongoDB	Explains horizontal scaling trade-offs, showing when to leverage highly flexible, fast document-based database systems.
PART IV: GENERATIVE AI & RAG RETRIEVAL (DAYS 9-10)		
Day 9	Generative AI: LLM Fundamentals & APIs ▪ Transformer architectures & attention overview ▪ Context windows & cost/latency trade-offs ▪ API key security & handling request timeouts ▪ Structured output via JSON mode	Transitions students from chatting with AI to programmatically integrating LLMs, ensuring outputs can drive backend application logic.
Day 10	Vector Databases, RAG & Prompting ▪ Text tokenization & high-dimensional vector embeddings ▪ Chroma DB & pgvector similarity search ▪ RAG pipeline: chunking text and query context injection ▪ Mitigating model hallucinations using local knowledge stores	Teaches interns to provide proprietary local datasets as context to public LLMs, enabling semantic document searching without fine-tuning costs.
PART V: AGENTIC SYSTEMS, OPENCLAW & HERMES (DAYS 11-15)		
Day 11	Agentic Systems: Harness, Rules & Skills ▪ Defining Agent autonomy: Reason-Act (ReAct) loop ▪ Function calling: declaring tool	Introduces self-governing software components capable of analyzing feedback, selecting tools, and fixing errors dynamically.

	schemas to LLMs ▪ Defining sandbox tool boundaries ▪ Managing states across agent execution steps	
Day 12	Multi-Agent Workflows & Orchestrations ▪ Coordinating subagent hierarchies ▪ Low-code n8n workflow integrations ▪ Chaining multi-agent inputs/outputs ▪ Triggering agent loops via webhooks/emails	Teaches workflow automation, letting interns combine REST endpoints and agent decisions with zero manual boilerplate code.
Day 13	OpenClaw Agent Orchestration ▪ OpenClaw core architecture & agent configurations ▪ Declaring skills and instructions ▪ Writing Python execution scripts for agents ▪ Debugging agent logs & execution trace paths	Exposes interns to code-first agent scripting, letting them construct and debug custom agent classes on local filesystems.
Day 14	Hermes Agent & Local Model Inference ▪ Pulling and serving open-weights models (Ollama) ▪ Running Hermes/LLaMA locally on LaaS GPU nodes ▪ Local hardware constraints (VRAM, compute metrics) ▪ Local data security & private network environments	Shows how to run models locally on supercomputing nodes to ensure data privacy and bypass external API rate limits.
Day 15	Team Project Showcases & Demo Day ▪ Final product integration check ▪ Live showcases & prototype pitching ▪ Feedback panel & evaluation ▪ Program wrap-up & roadmapping	Consolidates the 15-day curriculum by forcing teams to pitch their working full-stack AI POC to peers, proving their engineering readiness.